

Webhooks

সহজ কথায় বলতে গেলে, **Webhooks (ওয়েবহুক)** হলো একটি অ্যাপ থেকে অন্য একটি অ্যাপে কোনো ঘটনা ঘটার সাথে সাথে স্বয়ংক্রিয়ভাবে তথ্য পাঠানোর একটি পদ্ধতি।

একে বলা হয় "**Reverse API**" বা "**HTTP Push**"। সাধারণ API-তে আপনি সার্ভারকে জিজ্ঞেস করেন "নতুন কিছু আছে কি?", আর Webhook-এর ক্ষেত্রে সার্ভার নিজেই আপনাকে নক দিয়ে বলে "এই যে, নতুন একটা ঘটনা ঘটেছে!"

১. একটি বাস্তব উদাহরণ (Real-life Example)

মনে করুন আপনি অনলাইন থেকে একটি পিৎজা অর্ডার করেছেন। পিৎজা কোথায় আছে তা জানার দুটি উপায় আছে:

- **API পদ্ধতি:** আপনি প্রতি ৫ মিনিট পর পর পিৎজা শপকে ফোন দিয়ে জিজ্ঞেস করছেন, "আমার পিৎজা কি বের হয়েছে?" (একে বলা হয় **Polling**)।
- **Webhook পদ্ধতি:** আপনি পিৎজা শপকে বলে রাখলেন, "যখন পিৎজা ডেলিভারির জন্য বের হবে, তখন আমাকে একটা এসএমএস দিও।" এখানে দোকানদার নিজে থেকে আপনাকে তথ্য পাঠাচ্ছে, এটাই হলো ওয়েবহুক।

২. Webhooks কীভাবে কাজ করে?

Webhook কাজ করার জন্য তিনটি জিনিস প্রয়োজন হয়:

1. **Event (ঘটনা):** যে ঘটনার কারণে ওয়েবহুক ট্রিগার হবে (যেমন: কেউ পেমেন্ট করেছে, বা নতুন মেসেজ এসেছে)।
2. **Payload (তথ্য):** ওই ঘটনা সম্পর্কিত তথ্য (সাধারণত JSON ফরম্যাটে থাকে)।
3. **URL (ঠিকানা):** আপনার সার্ভারের একটি লিংক যেখানে ওই তথ্যটি পাঠানো হবে (একে বলা হয় **Webhook URL** বা **Endpoint**)।

৩. Webhooks বনাম API (পার্থক্য)

বিষয়	API (Polling)	Webhooks
কাজের ধরন	আপনি সার্ভার থেকে তথ্য টেনে আনেন (Pull)।	সার্ভার আপনার কাছে তথ্য পাঠিয়ে দেয় (Push)।
দক্ষতা	বারবার রিকোয়েস্ট পাঠাতে হয় বলে রিসোর্স বেশি খরচ হয়।	শুধু ঘটনা ঘটলেই তথ্য আসে, তাই অনেক সাশ্রয়ী।
সময়	তথ্য পেতে দেরি হতে পারে (Real-time নয়)।	সাথে সাথে তথ্য পাওয়া যায় (Real-time)।

৪. Webhooks-এর কিছু কমন ব্যবহার

- **Payment Gateways:** যেমন bKash, SSLCommerz বা Stripe-এ যখন কোনো কাস্টমার পেমেন্ট সফলভাবে শেষ করে, তখন পেমেন্ট প্রোভাইডার আপনার ওয়েবসাইটকে ওয়েবহুকের মাধ্যমে জানায় যে "পেমেন্ট সাফল্য হয়েছে, এখন অর্ডার কনফার্ম করো"।
- **GitHub/GitLab:** আপনি যখন কোড পুশ করেন, GitHub একটি ওয়েবহুক পাঠিয়ে আপনার CI/CD পাইপলাইন চালু করে দেয়।
- **Messaging Apps:** Slack বা Discord-এ নতুন কোনো নোটিফিকেশন বা মেসেজ আসলে ওয়েবহুক ব্যবহার করা হয়।

৫. Webhooks ব্যবহারের ঝুঁকি ও নিরাপত্তা

যেহেতু ওয়েবহুক একটি সাধারণ URL-এ ডাটা পাঠায়, তাই যে কেউ চাইলে ভুয়া ডাটা পাঠাতে পারে। এটি ঠেকানোর জন্য কিছু পদ্ধতি ব্যবহার করা হয়:

- **Secret Token:** আপনার এবং সার্ভিস প্রোভাইডারের মধ্যে একটি গোপন কোড থাকে যা দিয়ে ভেরিফাই করা হয় ডাটাটি সঠিক উৎস থেকে এসেছে কিনা।
- **HTTPS:** সবসময় সিকিউর কানেকশন ব্যবহার করা হয় যাতে মাঝপথে কেউ ডাটা চুরি করতে না পারে।

৬. সংক্ষেপে সারমর্ম

- **Webhook** হলো একটি অটোমেটেড সিস্টেম যা রিয়েল-টাইমে ডাটা আদান-প্রদান করে।
- এটি তখনই কাজ করে যখন নির্দিষ্ট কোনো **Event** ঘটে।
- এটি ইন্টারনেটের আধুনিক অ্যাপগুলোকে একে অপরের সাথে কথা বলতে সাহায্য করে।

Webhook কী?

Webhook = Event-based HTTP callback

সহজ ভাষায় 

“কিছু একটা ঘটলেই আমাকে জানাও”

 Server নিজে থেকেই **POST request পাঠায়**

 Real-life analogy:

- Food delivery app: *Order delivered* → notification
- YouTube: *New video* → subscriber alert

Webhook কেন দরকার?

আগে  (Polling)

- বারবার API hit
- Slow
- Costly

Webhook এ 

- Real-time
- Low latency
- Low cost
- Event driven

Polling vs Webhook

Polling	Webhook
---------	---------

বারবার request	Event হলে request
----------------	-------------------

Slow	Fast
------	------

Cost বেশি	Efficient
-----------	-----------

Webhook Architecture

Event happens

↓

Source Server

↓ HTTP POST

Webhook URL (Your Server)



Process / Store / Trigger action

Webhook Flow (Step by Step)

1. তুমি একটা **Webhook URL** বানাও
2. Third-party কে দাও (Stripe, GitHub)
3. Event ঘটলে তারা **POST request** পাঠায়
4. তুমি receive করে process করো

Simple Webhook Example

Sender (Third-party)

POST /webhook

Content-Type: application/json

```
{  
  "event": "payment_success",  
  "amount": 500  
}
```

Receiver (Node.js Express)

```
app.post("/webhook", (req, res) => {  
  console.log(req.body);  
})
```

```
res.status(200).send("OK");  
});
```

Real-World Webhook Use Cases

- Stripe → Payment success/fail
- GitHub → Push, PR event
- Razorpay → Refund
- Slack → Message event
- CI/CD → Build status

Webhook Security (Very Important 🔥)

1. Secret / Signature verification ★

HMAC signature match

👉 Prevent fake requests

2. HTTPS only

👉 Data encrypted

3. IP Whitelisting

👉 Only trusted source allowed

4. Replay attack protection

👉 Timestamp + nonce

🧠 Idempotency (Interview favorite)

👉 Same webhook multiple times আসতে পারে

Fix:

event_id already processed? → ignore

🕒 Retry Mechanism

Webhook fail হলে:

- Retry with exponential backoff
- Queue system (SQS, RabbitMQ)

🔗 Sync vs Async Webhook

Sync

Async

Immediate response Background process

Slow risk

Scalable

👉 Best practice: **ACK fast, process async**

📦 Webhooks + Queue Architecture

Webhook

↓

API

↓

Queue

↓

Worker

↓

DB / Email / Notification



Webhook vs API

API	Webhook
Client pulls	Server pushes
Request based	Event based



Interview Must-Know Q&A

Q: Why webhook instead of polling?



Real-time & efficient

Q: How secure webhook?



Signature + HTTPS

Q: What if webhook sent twice?



Idempotency

Q: Webhook timeout issue?



Queue + async

System Design Example

Payment System

- Stripe webhook
- Verify signature
- Store event
- Update order status
- Send email

Best Practices

- ✓ Always verify signature
- ✓ Respond 200 quickly
- ✓ Use queue
- ✓ Log everything
- ✓ Retry safe

One-Line Summary

Webhook = Event → HTTP → Action

Polling is waste, Webhook is smart 🤓